

Graph Visualization

Prof. O-Joun Lee

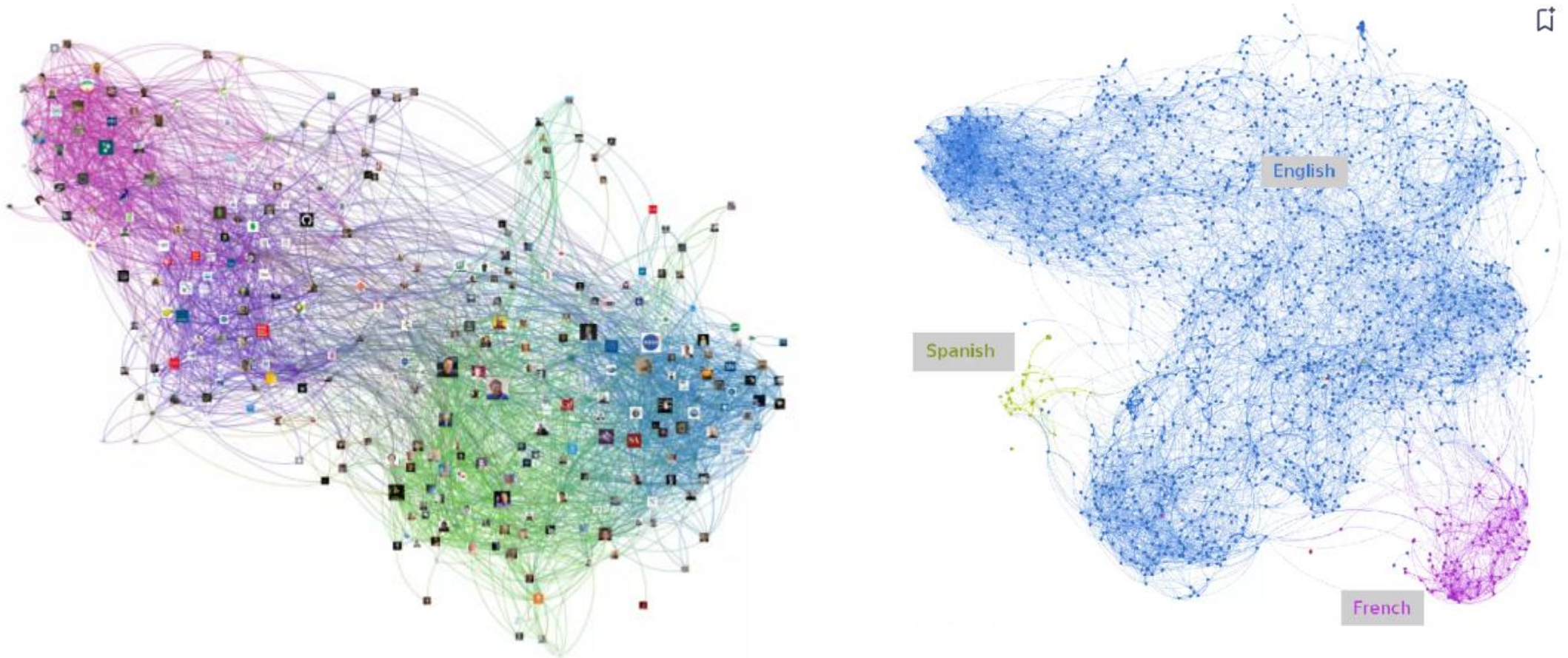
Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents

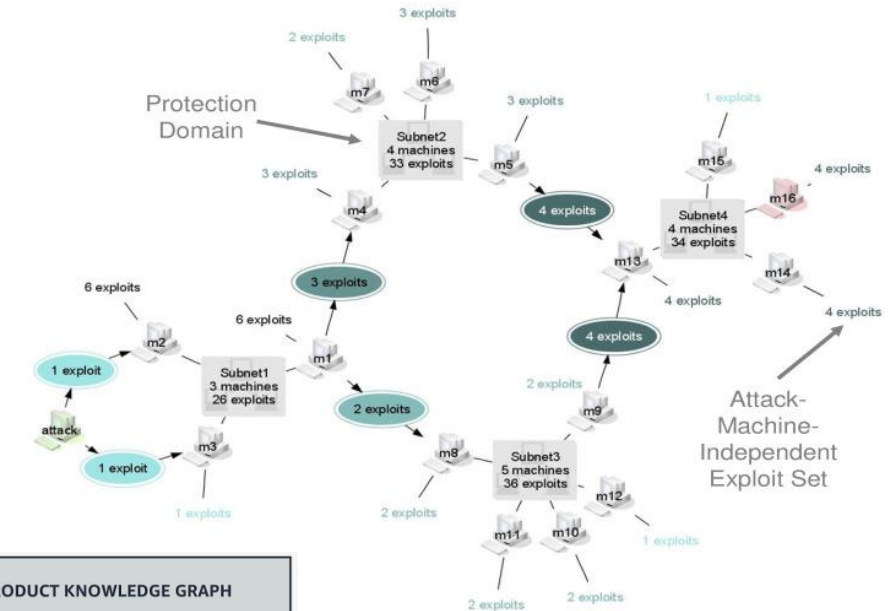
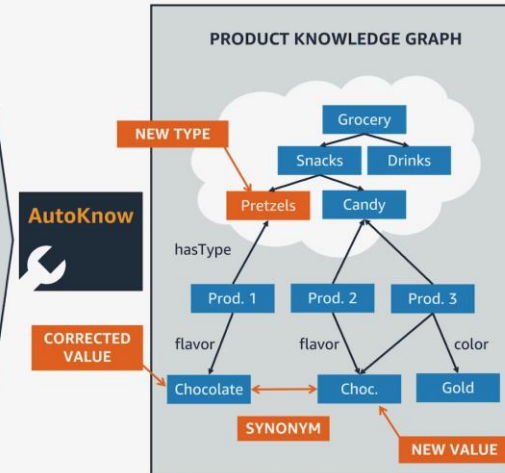
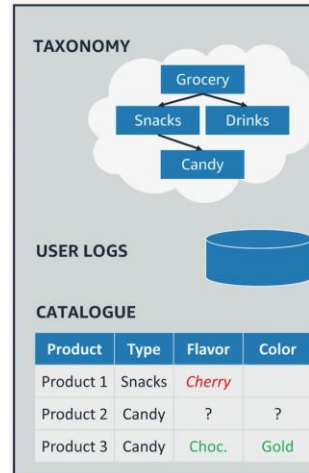
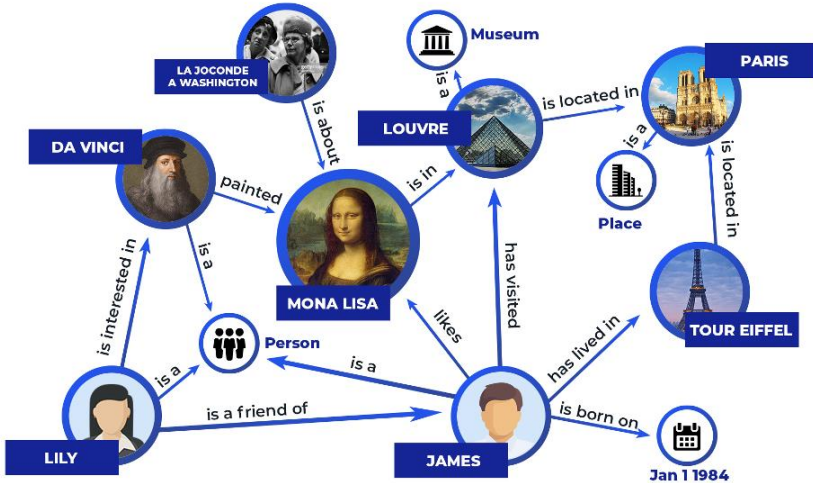


- Graph visualization:
 - Visualization techniques:
 - Spring-embedded
 - Circular, etc.
 - Tools for graph exploration and visualization:
 - Gephi, Cytoscape, etc.

- A way of representing structural relationships between objects as diagrams
- Use nodes as objects and edges to connect between them
- Illustrate relationships and patterns in various fields like social networks,...



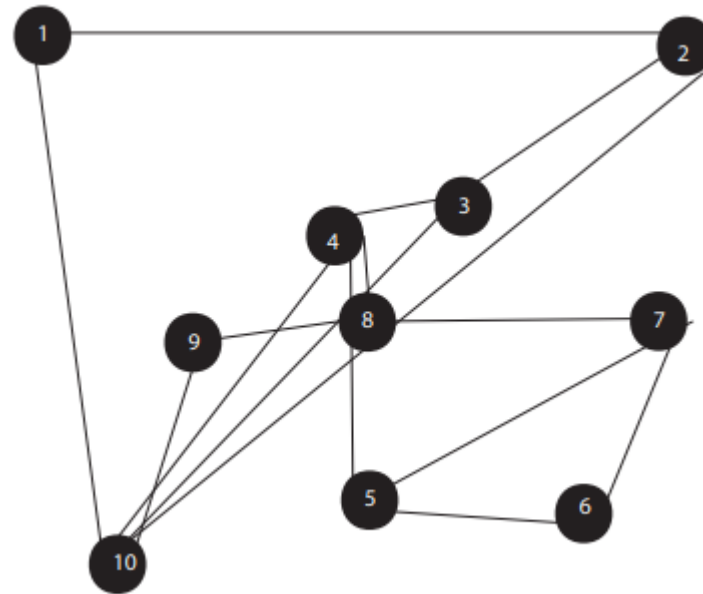
- Knowledge Graph
- Fraud Detection (e.g. banking)
- Cybersecurity (Attack graph)
- Recommendation system (Netflix, Amazon, LinkedIn,...)
- Biological network (e.g. drug discovery, protein interaction,...)



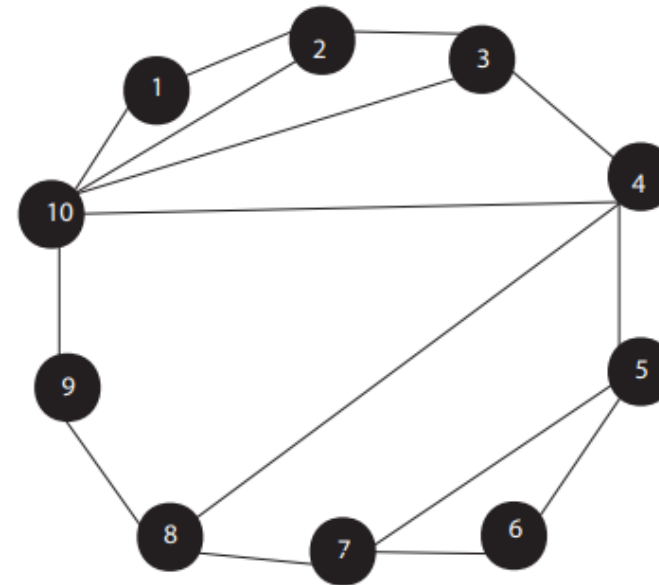
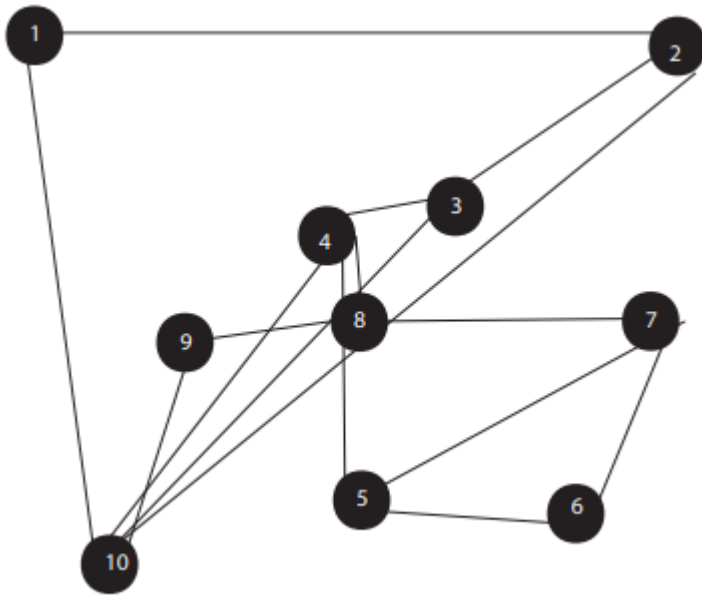
Sources:

- PuppyGraph
- ml-digest
- Sridevi, M., Rao, R. R., & Rao, M. V. (2016). A survey on recommender system. International Journal of Computer Science and Information Security, 14(5), 265.

➤ **Input:** Graph $G = (V, E)$



- **Input:** Graph $G = (V, E)$
- **Output:** Clear and readable drawing of G



➡ Which criteria would you optimize?

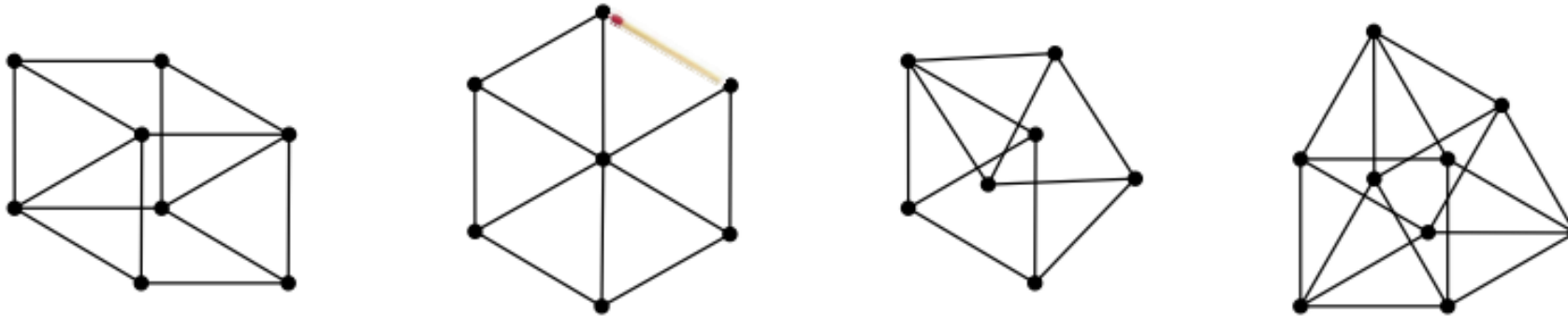
-

- **Input:** Graph $G = (V, E)$
- **Output:** Creating clear and readable drawings of graph G .
- **Criteria:**
 - Adjacent nodes are close.
 - Non-adjacent nodes are far.
 - The preservation of edge length: edges short, straight-line, **similar length**.
 - Densely connected nodes tend to close.
 - Draw G with as few crossings as possible.
 - Nodes distributed evenly.

Let's take an example

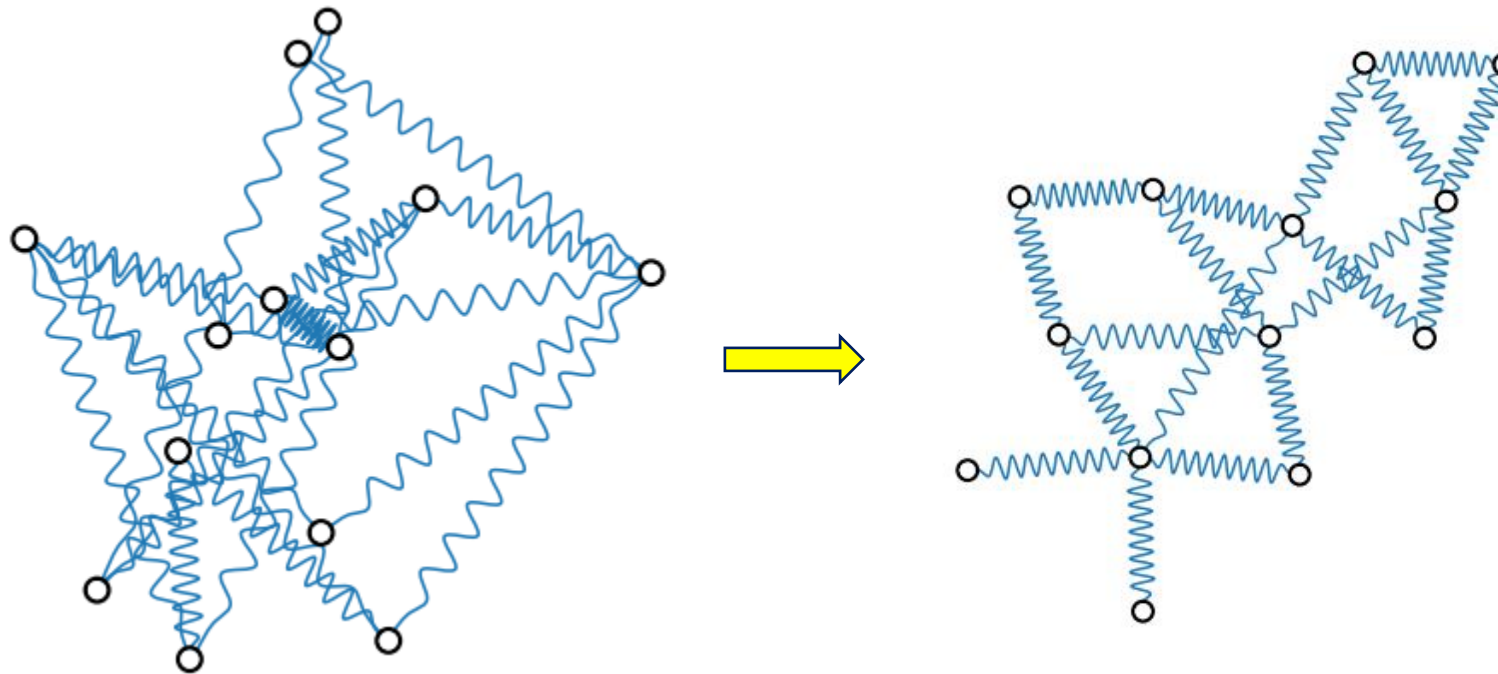


- **Input** : Graph $G = (V, E)$, required edge length $l(e), \forall e \in E$
- **Output**: Drawing of G which realizes all the edge lengths

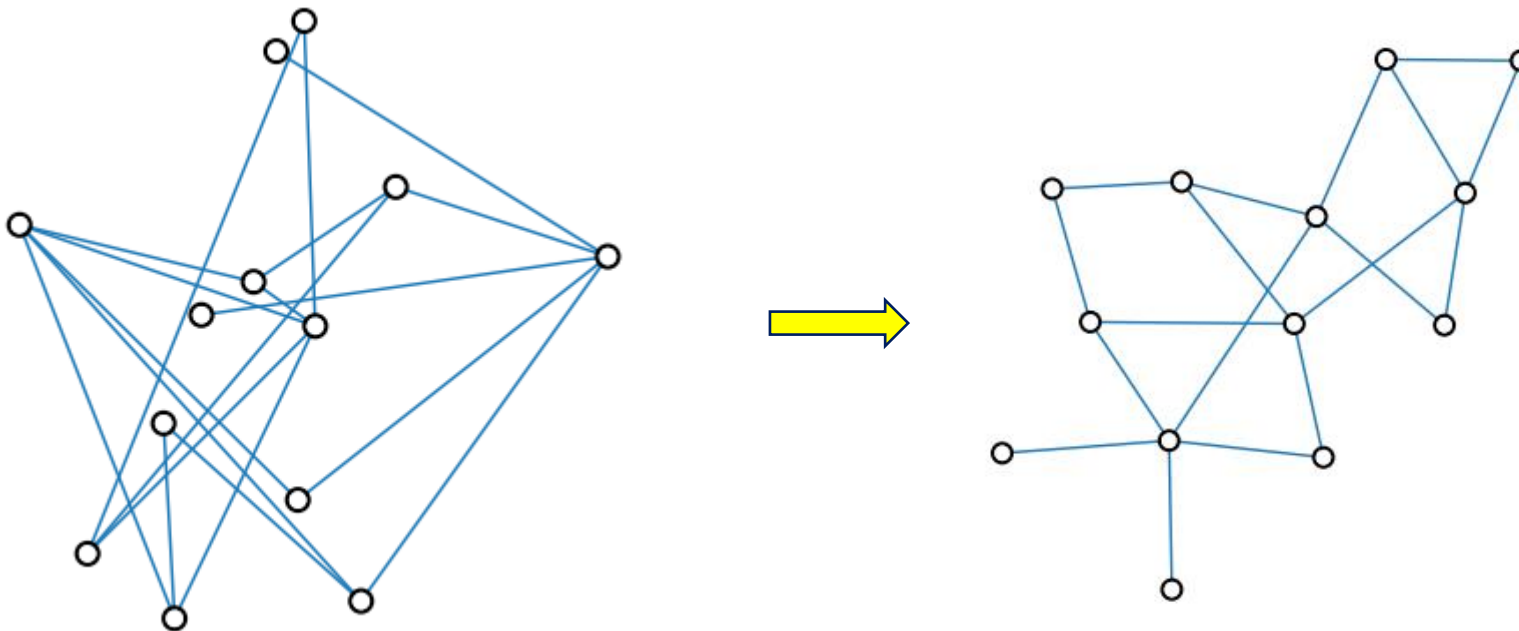


- **NP-hard problem** for:
 - Uniform edge lengths in any dimension
 - Uniform edge lengths in planar drawing

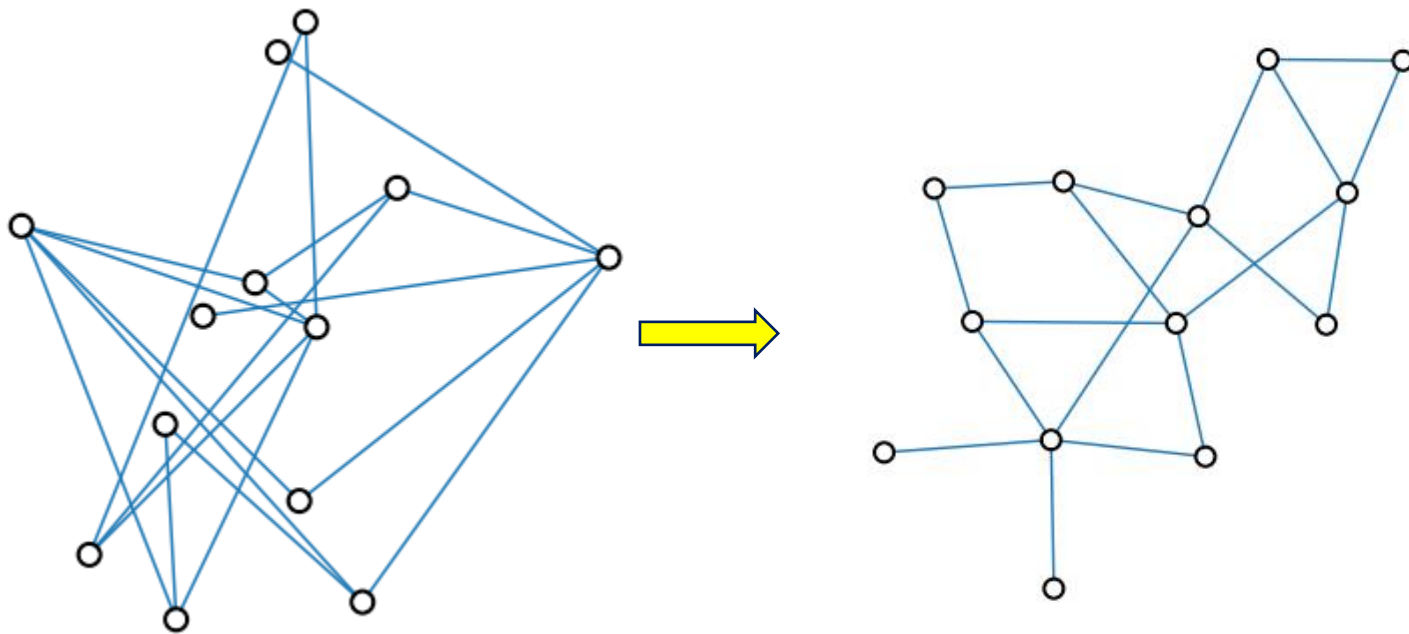
- To embed a graph, we replace the vertices by steel rings and replace each edge with a spring to form a mechanical system
 - The nodes are placed in some initial layout
 - The spring forces on the rings move the system to a minimal energy state



- To embed a graph, we replace the vertices by steel rings and replace each edge with a spring to form a mechanical system
 - The nodes are placed in some initial layout
 - The spring forces on the rings move the system to a minimal energy state



- To embed a graph, we replace the vertices by steel rings and replace each edge with a spring to form a mechanical system
 - The nodes are placed in some initial layout
 - The spring forces on the rings move the system to a minimal energy state



- Adjacent nodes u and v : f_{spring}



- Repulsive forces:
non-adjacent nodes u and v : f_{rep}



- Repulsive force between two non-adjacent node pairs v_i and v_j

$$f_{rep}(p_i, p_j) = \frac{c_{rep}}{||p_i - p_j||^2} \cdot p_i \vec{p}_j$$

- Attractive force between two adjacent vertices v_i and v_j

$$f_{spring}(p_i, p_j) = c_{spring} \log \frac{||p_i - p_j||}{l} \cdot p_i \vec{p}_j$$

- Resulting displacement vector for node v_i

$$F_i(t) \leftarrow \sum_{(v_i, v_j) \notin E} f_{rep}(p_j, p_i) + \sum_{(v_i, v_j) \in E} f_{spring}(p_j, p_i)$$

Where:

- $l = l(e)$: the ideal spring length of edge e .
- $||p_i - p_j||$: Distance between v_i and v_j .
- $p_i \vec{p}_j$: unit vector pointing from v_i to v_j .
- c_{rep} : repulsion constant (e.g. 1.0).
- c_{spring} : spring constant (e.g. 2.0)

Initial layout with random positions of nodes in the layout

Algorithm 1: SpringEmbedder

$G = (V, E)$, $p = (p_i)$, $v_i \in V$, $\epsilon > 0$, $K \in N$

Input: p : initial layout, ϵ : threshold

Output: p : is end layout

1

2

3

4

5

6

7

8 Return p

End layout

- Spring forces:
Adjacent nodes u and v : f_{spring}
- Repulsive forces:
non-adjacent nodes u and v : f_{rep}

Initial layout with random positions of nodes in the layout

Algorithm 1: SpringEmbedder

$G = (V, E)$, $p = (p_i)$, $v_i \in V$, $\epsilon > 0$, $K \in N$

Input: p : initial layout, ϵ : threshold

Output: p : is end layout

```
1  $t \leftarrow 1$ 
2
3
4
5
6
7
8 Return  $p$ 
```

End layout

- Spring forces:
Adjacent nodes u and v : f_{spring}
- Repulsive forces:
non-adjacent nodes u and v : f_{rep}

Initial layout with random positions of nodes in the layout

Algorithm 1: SpringEmbedder

$G = (V, E)$, $p = (p_i)$, $v_i \in V$, $\epsilon > 0$, $K \in \mathbb{N}$

Input: p : initial layout, ϵ : threshold

Output: p : is end layout

```

1  $t \leftarrow 1$ 
2 while  $t < K$  and  $\max_{v_i \in V} \|F_i(t)\| > \epsilon$  do
3   for  $v \in V$  do
4      $F_i(t) \leftarrow \sum_{(v_i, v_j) \notin E} f_{rep}(p_j, p_i) + \sum_{(v_i, v_j) \in E} f_{spring}(p_j, p_i)$ 
5   for  $v \in V$  do
6      $p_i \leftarrow p_i + \delta(t) \cdot F_i(t)$ 
7    $t \leftarrow t + 1$ 
8 Return  $p$ 
    
```

Update new location of node

End layout

cooling factor

➤ Spring forces:

Adjacent nodes u and v : f_{spring}

➤ Repulsive forces:

non-adjacent nodes u and v : f_{rep}

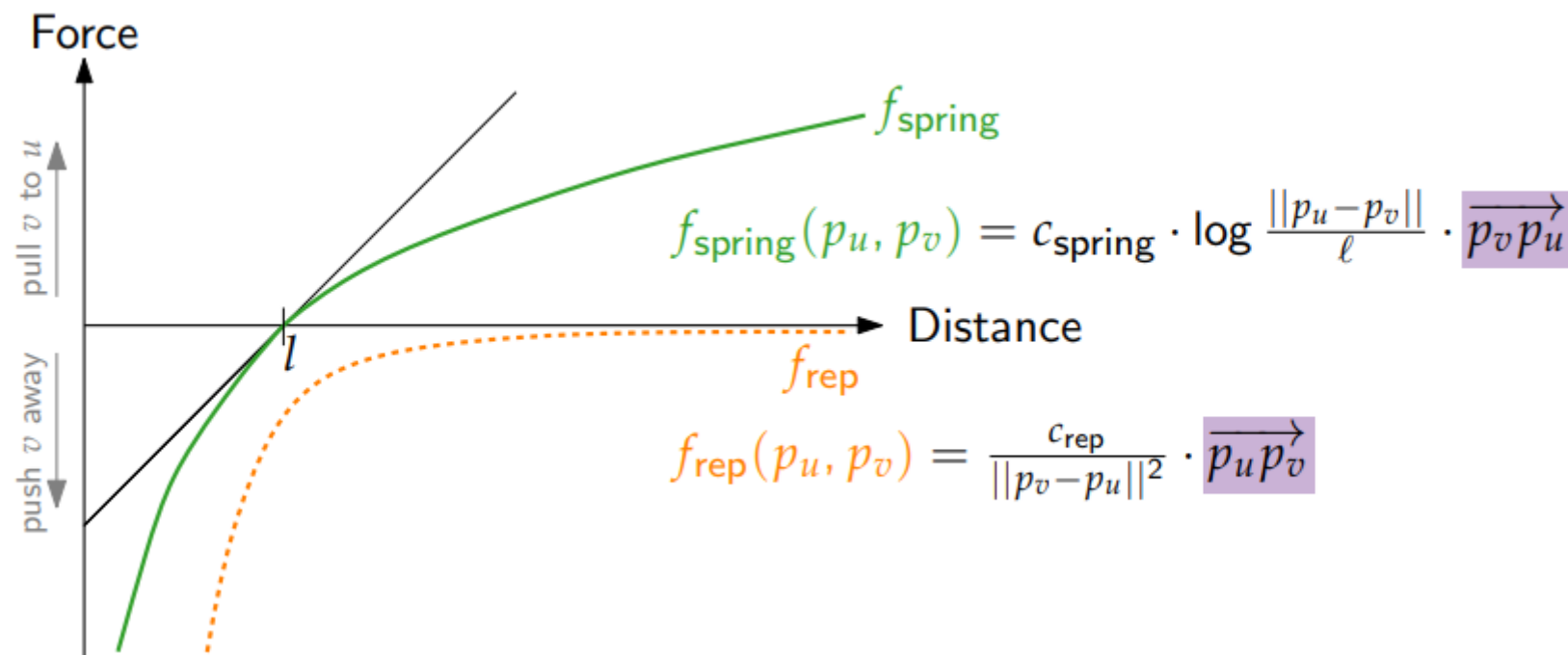
Where:

- $l = l(e)$: the ideal spring length of edge e .
- $\|p_i - p_j\|$: Distance between v_i and v_j .
- $p_i \vec{p}_j$: unit vector pointing from v_i to v_j .
- c_{rep} : repulsion constant (e.g. 1.0).
- c_{spring} : spring constant (e.g. 2.0)

$$f_{rep}(p_i, p_j) = \frac{c_{rep}}{\|p_i - p_j\|^2} \cdot p_i \vec{p}_j$$

$$f_{spring}(p_i, p_j) = c_{spring} \log \frac{\|p_i - p_j\|}{l} \cdot p_i \vec{p}_j$$

- Spring forces (f_{spring}): pull node v close to node u (u and v are adjacent)
- Repulsive forces (f_{rep}): push node v far away node u (u and v are non-adjacent)



➤ Advantages:

- Simple algorithm
- Good results for small and medium-sized graphs
- Good representation of symmetry and structure

➤ Disadvantages:

- System is not stable at the end
- Converging to local minimal

- Repulsive force between **all** vertex pairs v_i and v_j

$$f_{rep}(p_i, p_j) = \frac{l}{||p_i - p_j||^2} \cdot p_i \vec{p}_j$$

- Attractive force between two adjacent vertices v_i and v_j

$$f_{attactive}(p_i, p_j) = \frac{||p_i - p_j||^2}{l} \cdot p_i \vec{p}_j$$

- Resulting force between adjacent vertices v_i and v_j

$$f_{spring}(p_i, p_j) = f_{rep}(p_i, p_j) + f_{attactive}(p_i, p_j)$$

Algorithm 1: SpringEmbedder

$G = (V, E), p = (p_i), v_i \in V, \epsilon > 0, K \in \mathbb{N}$

Input: p : initial layout, ϵ : threshold

Output: p : is end layout

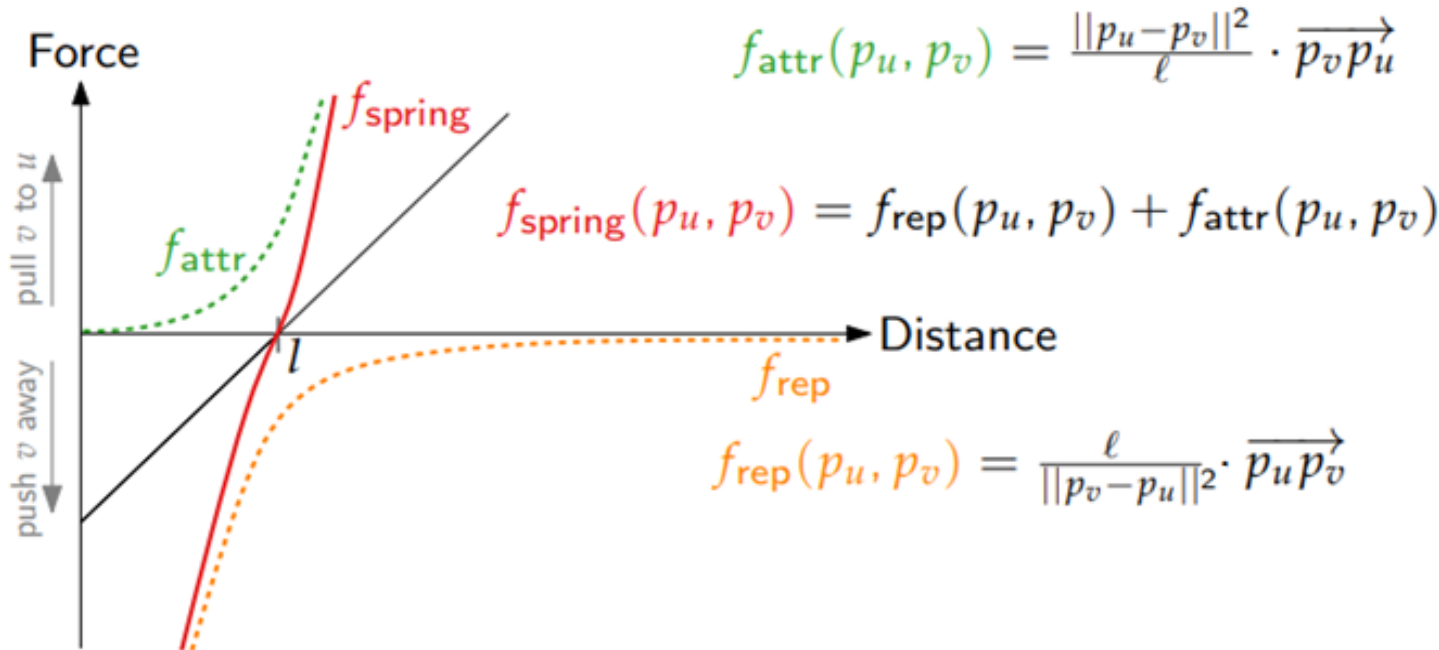
```

1  $t \leftarrow 1$ 
2 while  $t < K$  and  $\text{MAX}_{v_i \in V} ||F_i(t)|| > \epsilon$  do
3   for  $v \in V$  do
4      $F_i(t) \leftarrow \sum_{(v_i, v_j) \notin E} f_{rep}(p_j, p_i) + \sum_{(v_i, v_j) \in E} f_{spring}(p_j, p_i)$ 
5   for  $v \in V$  do
6      $p_i \leftarrow p_i + \delta(t) \cdot F_i(t)$ 
7    $t \leftarrow t + 1$ 
8 Return  $p$ 

```

There are three forces:

- Spring forces (f_{spring}): pull node v close to node u (u and v are adjacent).
- Attractive force between two adjacent nodes v_i and v_j (f_{attr}): pull node v close to node u (u and v are adjacent).
- Repulsive forces (f_{rep}): push node v faraway node u (u and v are non-adjacent).



➤ Spring layout

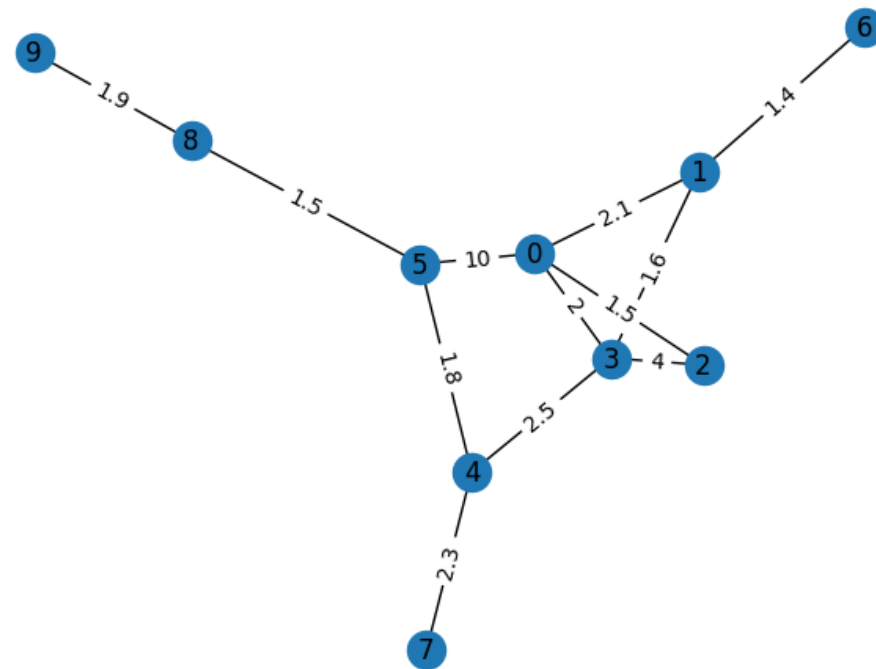
```
# Instantiate the graph
G = nx.Graph()

edges = [(0, 1, 2.1), (0, 2, 1.5), (0, 3, 2), (0, 5, 10), (1, 3, 1.6), (1, 6, 1.4),
         (2, 3, 4), (3, 4, 2.5), (4, 5, 1.8), (4, 7, 2.3), (5, 8, 1.5), (8, 9, 1.9)]

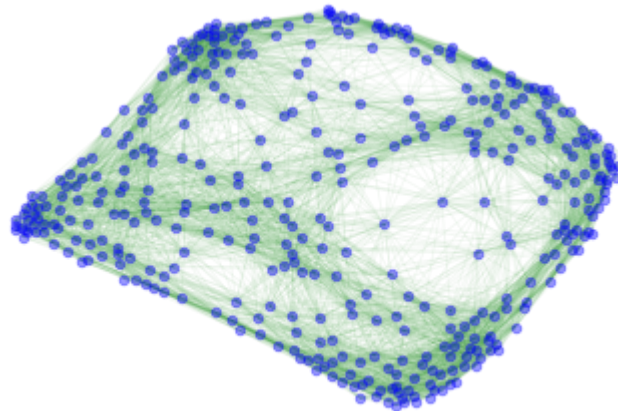
# add node/edge pairs
G.add_weighted_edges_from(edges)

pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels = True)

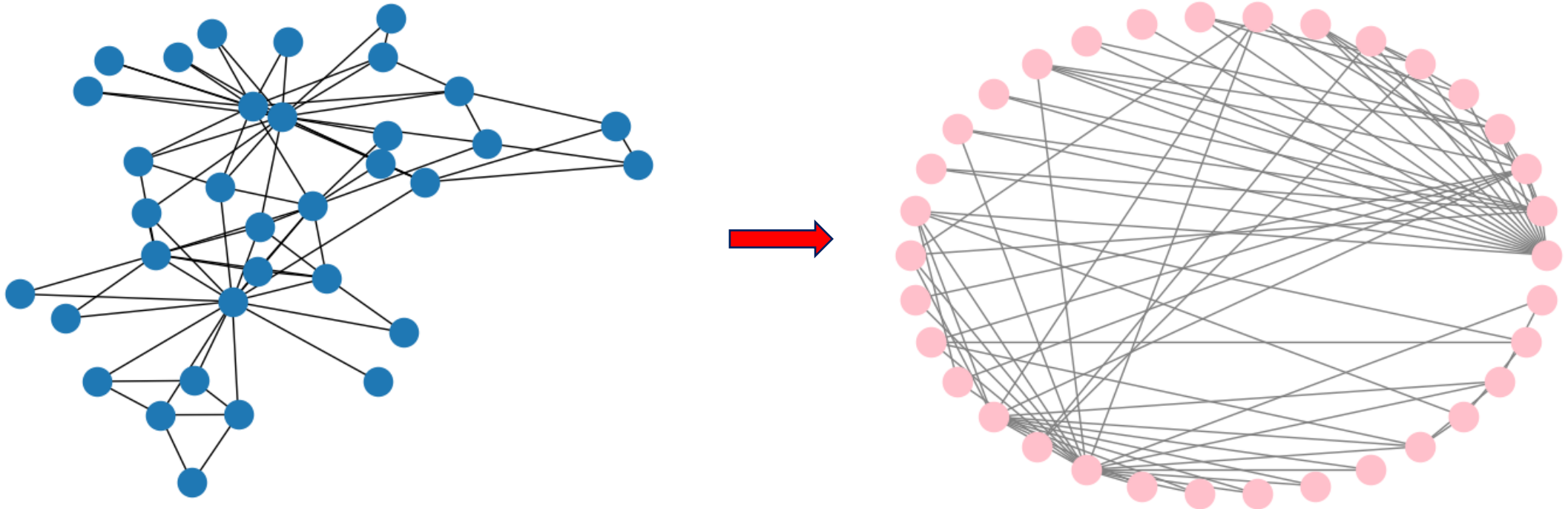
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
```



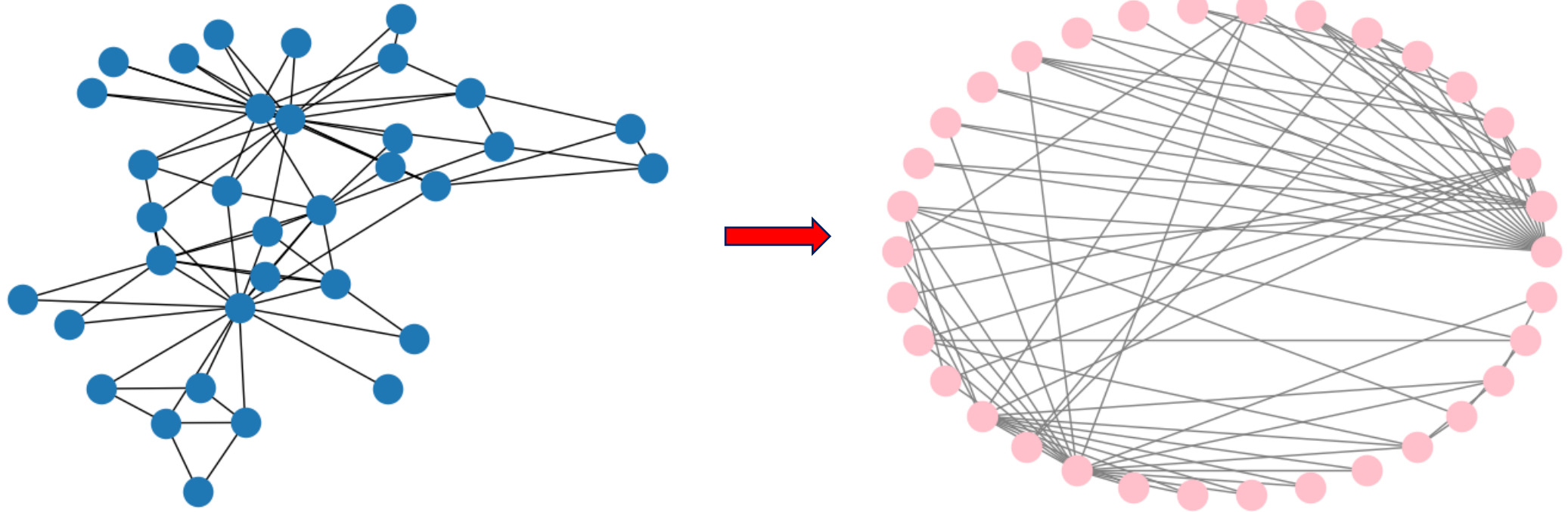
- ForceAtlas2 is a fast, force-directed graph layout algorithm designed for visualizing large-scale networks (10,000+ nodes), famously used in Gephi.
- It spatializes graphs by simulating physical forces - repulsing nodes like charged particles and attracting them via edges. It is particularly effective for detecting community structures and network clusters.
- Key Features & Characteristics
 - **Performance:** Faster than traditional algorithms like *Fruchterman-Reingold*.
 - **Continuous Layout:** Allows for real-time interaction and refinement of network layouts.
 - **Balance:** Manages "temperature" to reduce oscillation of nodes, allowing them to settle into stable positions.
 - **Implementation:** Originally in Java for Gephi, it is available for Python (NetworkX/igraph) and JavaScript ([graphology](https://github.com/d3.js/d3-force)).



- **Input:** Graph $G = (V, E)$
- **Output:** Creating circular drawings of graph G .



- **Input:** A biconnected graph, $G = (V, E)$.
- **Output:** A circular drawing Γ of G such that each node in V lies on the periphery of a single embedding circle.



➤ In circular graphs, close nodes should not be connected:

➤ Idea: Finding and store nodes that have two non-connected neighbours by using BFS algorithm.

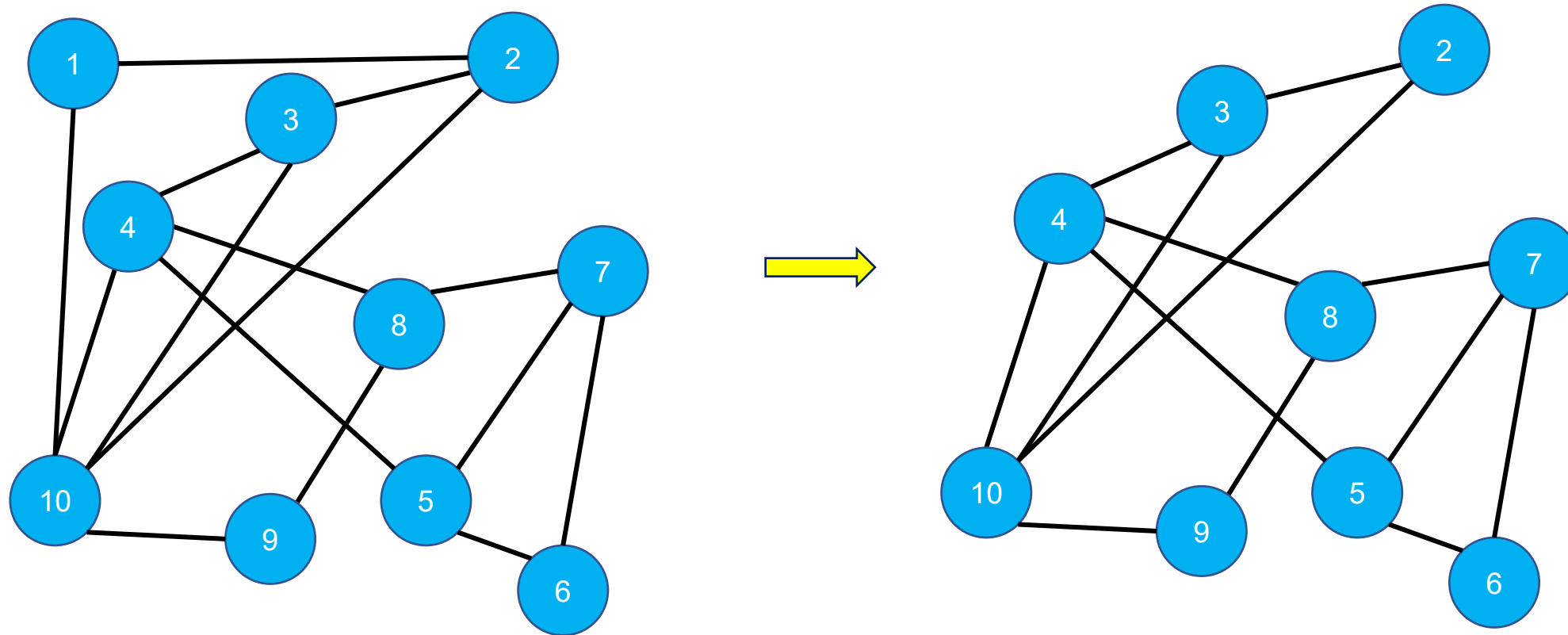
➤ Implement:

➤ Starting at a random node, store nodes that do not have non-connected neighbours in a stack.

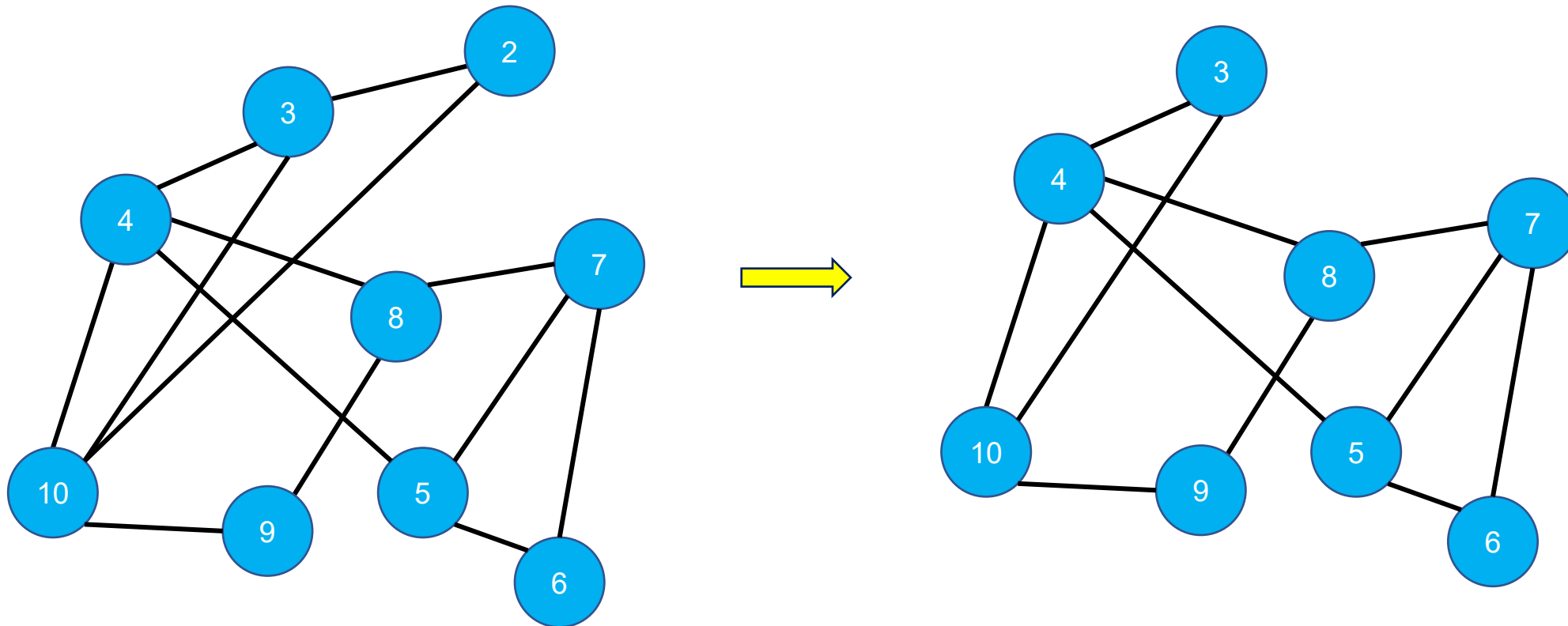
➤ Restore the graph to generate a circle graph

1. Bucket sort the nodes by ascending degree into a table T .
2. Set $counter$ to 1.
3. While $counter \leq n - 3$
4. If a wave front node u has lowest degree then $currentNode = u$.
5. Else If a wave center node v has lowest degree then
 $currentNode = v$.
6. Else set $currentNode$ to be some node with lowest degree.
7. Visit the adjacent nodes consecutively. For each two nodes,
8. If a pair edge exists place the edge into $removalList$.
9. Else place a triangulation edge between the current pair of
 neighbors and also into $removalList$.
10. Update the location of $currentNode$'s neighbors in T .
11. Remove $currentNode$ and incident edges from G .
12. Increment $counter$ by 1.
13. Restore G to its original topology.
14. Remove the edges in $removalList$ from G .
15. Perform a DFS (or a longest path heuristic) on G .
16. Place the resulting longest path onto the embedding circle.
17. If there are any nodes which have not been placed then place the remaining nodes into the embedding order with the following priority:
 (i) between two neighbors, (ii) next to one neighbor, (iii) next to zero neighbors.

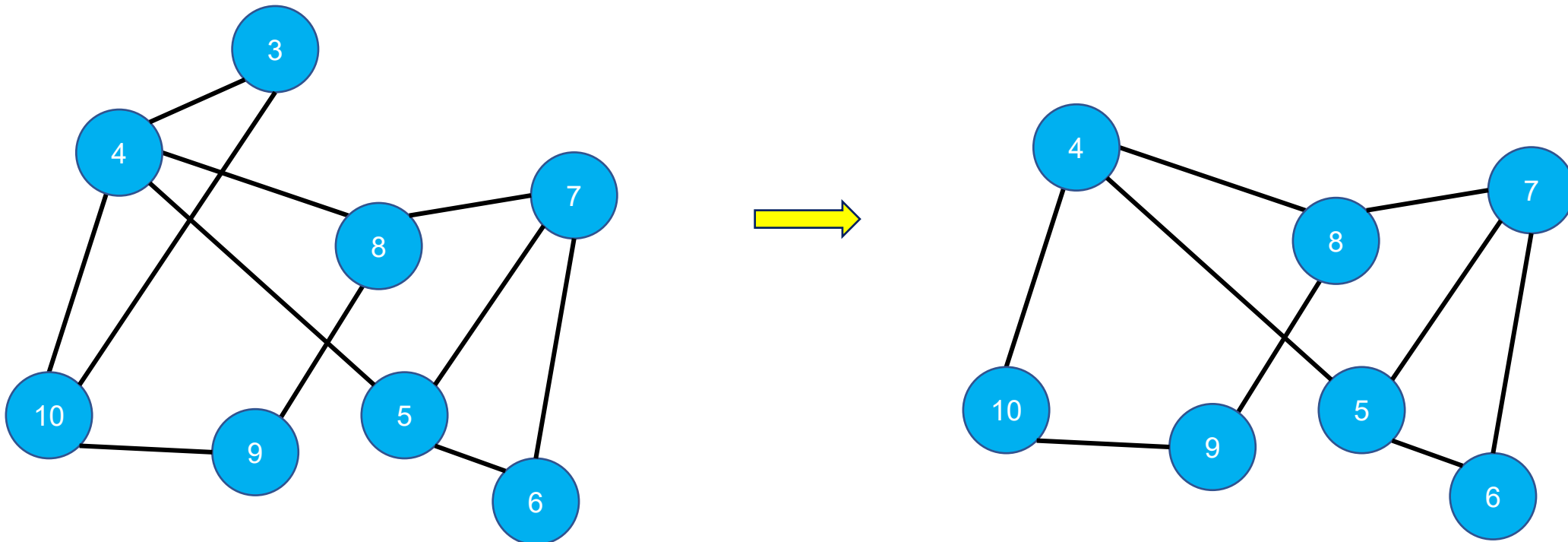
- Start with a graph G in the left side
- First, we choose randomly Node 1
- We check for edge(2,10), which exists. We store it and remove Node 1.



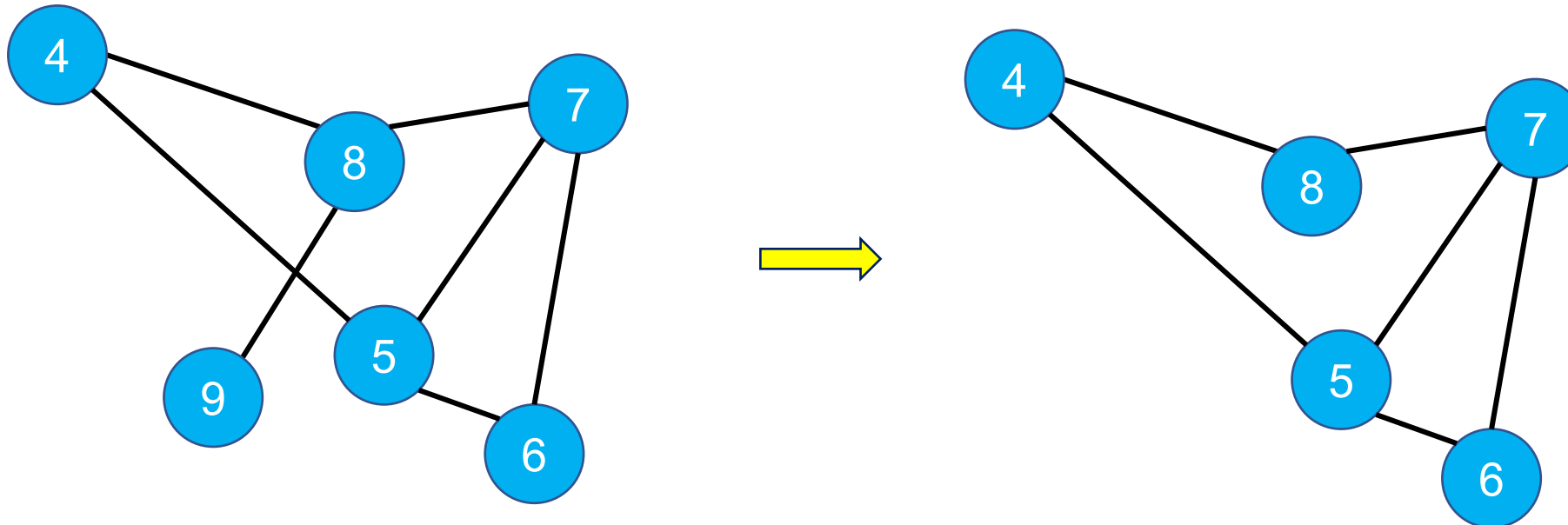
- Next, we choose a lowest-degree neighbor of the removed Node 1, which is 2.
- Check for edge (3,10) which exists. We store it and remove Node 2.



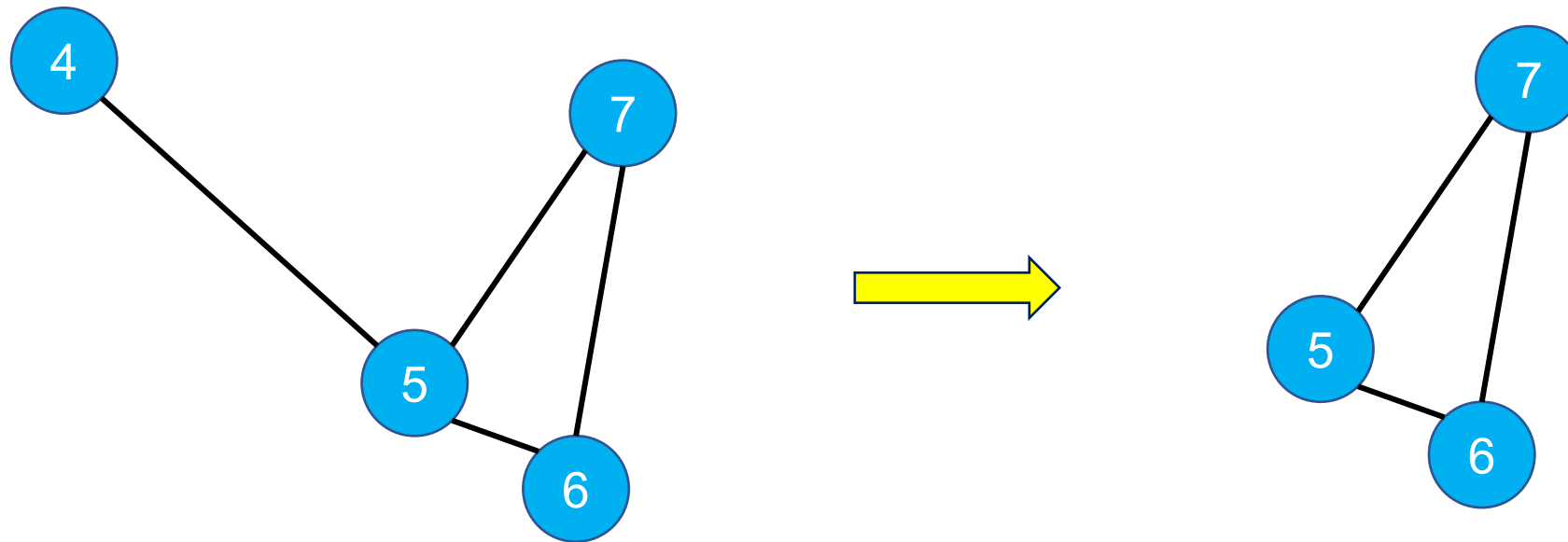
- Next, we select a lowest-degree neighbor of Node 2.
 - This is Node 3. We check for edge (4,10).
- It exists so we store it and remove Node 3.



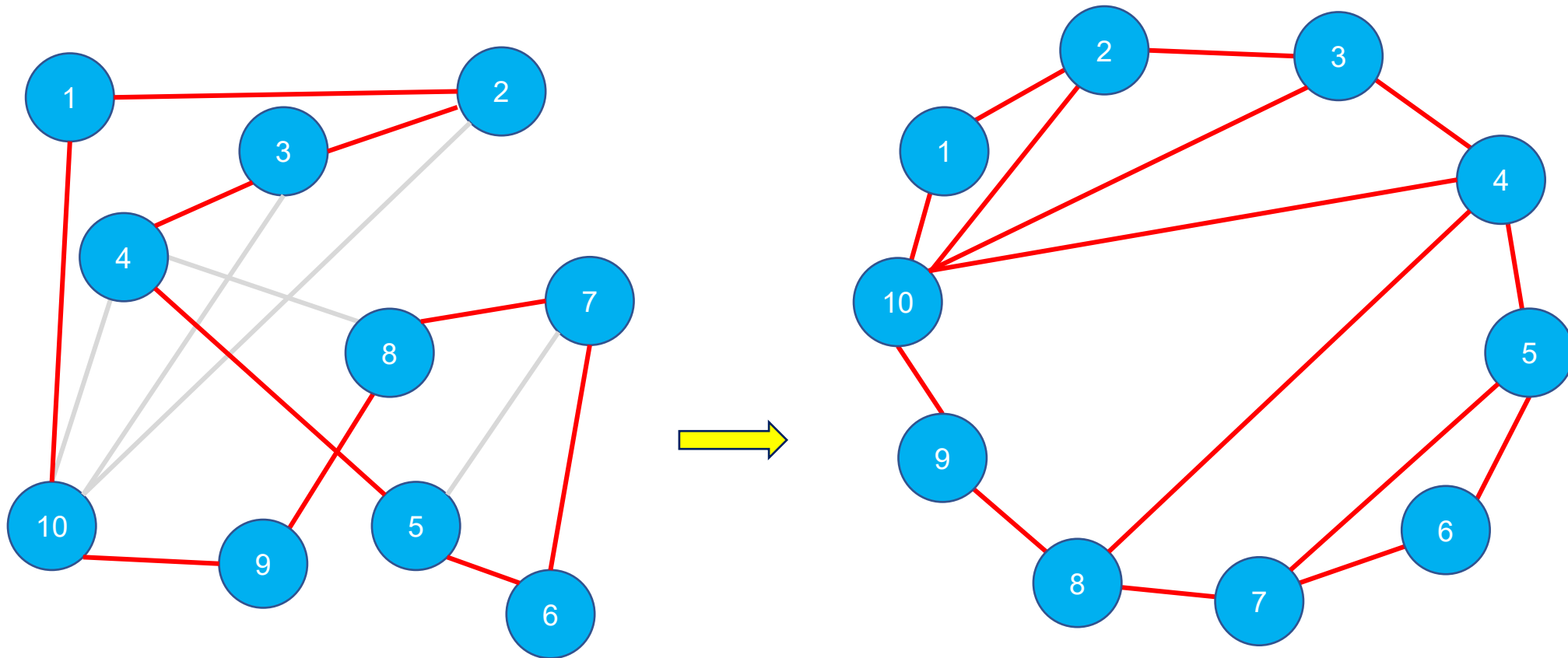
- Similarly, we can select Node 10 and check for edge (4,9).
 - It does not exist.
 - So, we add edge (4,9) which is a triangulation edge, store it and remove Node 10.
- We continue choosing Node 9 and check for edge (4,8). It exists so we store it and remove Node 9. Next, for Node 8 we check for edge(4,7) which does not exist. We add it to the graph and store it. After this, we remove Node 8.



- In the same way, we select vertex 4 and check for edge (5,7), which exists.
 - So, we mark.
- Now we have only three vertices left, so this phase of the algorithm is completed.



- Now we restore the graph and remove all stored edges.
- Since the graph is outerplanar, we have the Hamilton circle left.



➤ Circular layout

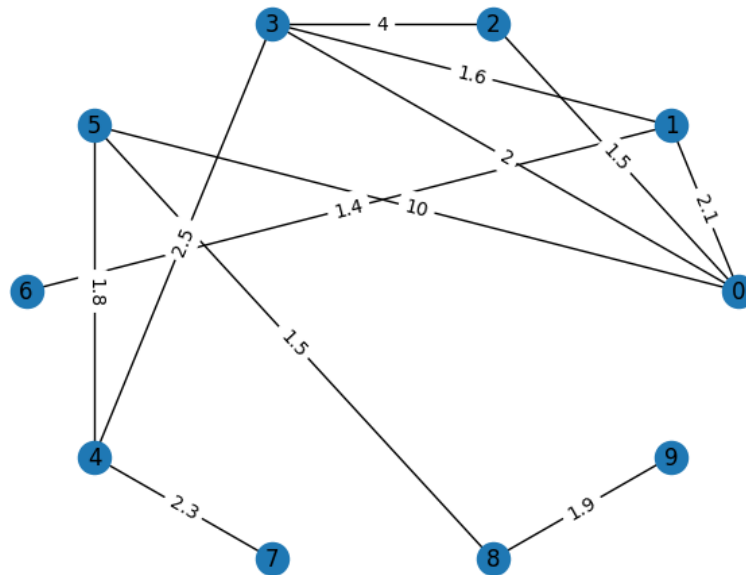
```
# Instantiate the graph
G = nx.Graph()

edges = [(0, 1, 2.1), (0, 2, 1.5), (0, 3, 2), (0, 5, 10), (1, 3, 1.6), (1, 6, 1.4),
         (2, 3, 4), (3, 4, 2.5), (4, 5, 1.8), (4, 7, 2.3), (5, 8, 1.5), (8, 9, 1.9)]

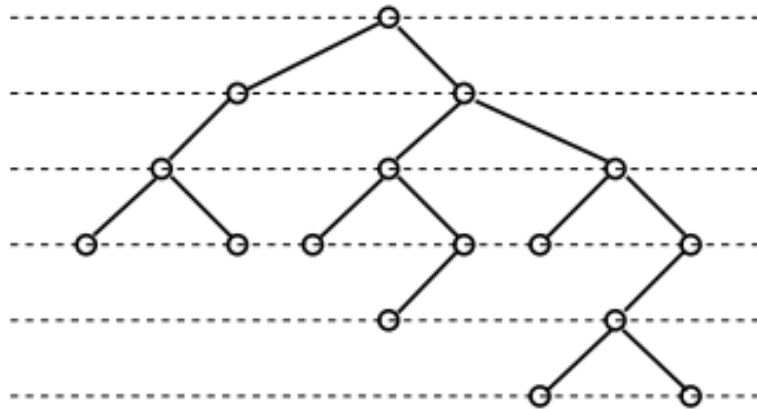
# add node/edge pairs
G.add_weighted_edges_from(edges)

pos = nx.circular_layout(G)
nx.draw(G, pos, with_labels = True)

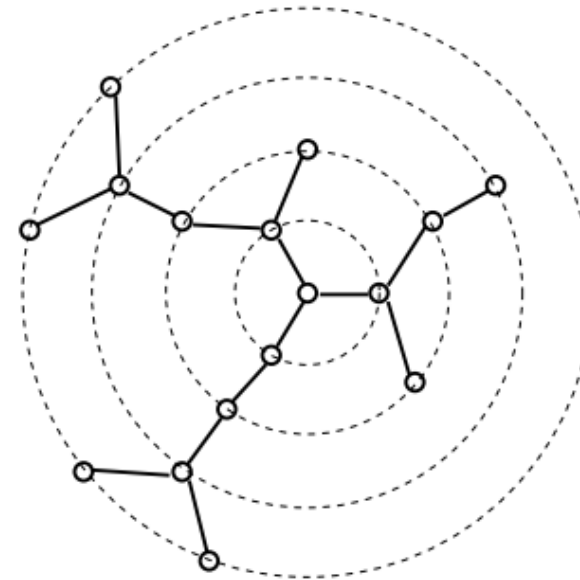
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
```



- Trees:
 - Requirements:
 - No two edges cross.
 - A child should be placed below its parent in the y-direction.
 - Strongly order-preserving drawing.

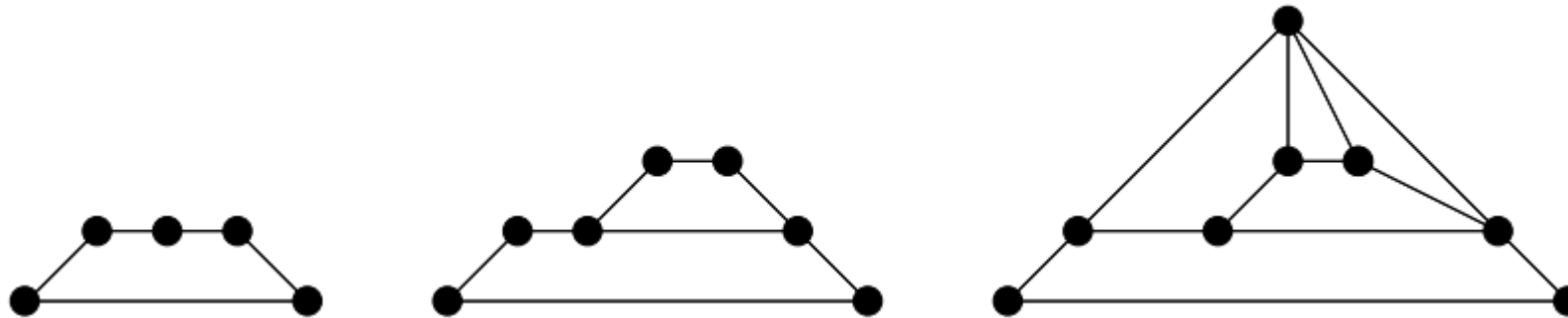


A layered tree drawing



A radial tree drawing

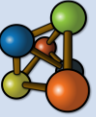
- Given an input graph $G = (V, E)$:
 - Kant used the canonical ordering approach to develop straight-line algorithm
 - The algorithm aims to form a chain and give them the same y-coordinate



An example for the straight-line algorithm of Kant

➤ There are several open-source tools for network analysis:

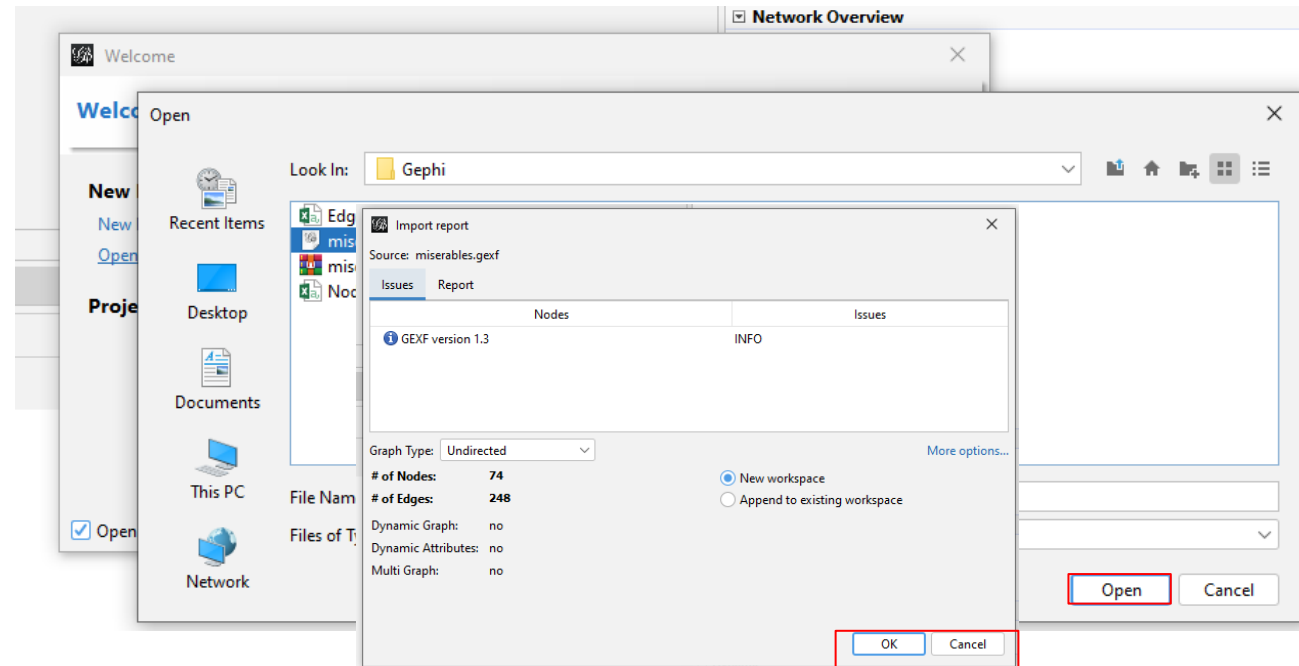
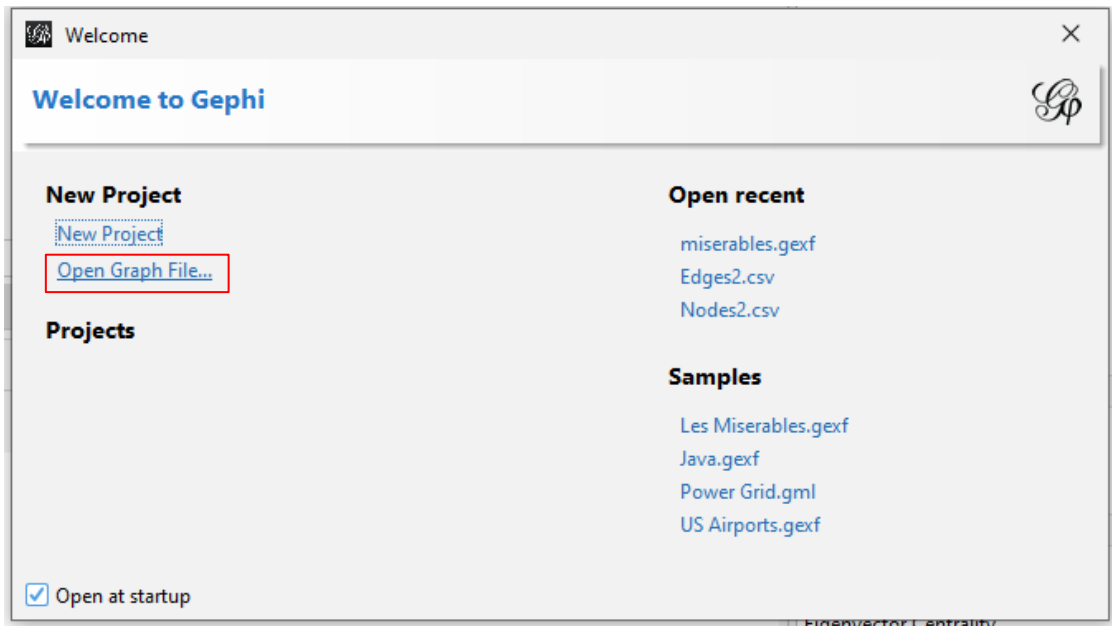
- NetworkX in Python
- Gephisto
- iGraph packages in R
- Ucinet
- Gephi
- Graphviz
- Cytoscape
- Pyvis
- NodeXL
- Plotly
- Graphia
- Etc...

	Exploration tools Interactively explore graphs	Development tools For working with graph data	Dashboarding tools Build graph dashboards	Embedded tools Embed graphs in your apps
Neo4j tooling	 Neo4j Bloom	 Neo4j Brower	 NeoDash	 NVL
Open-source tools	  NODE	 ipysigma  yWorks for Jupyter		 React-force-graph  Cytoscape  Graphviz
Enterprise tools	 Linkurious  Hume  Graphia  SemSpect		 GRAPHILEON	 Cosmograph  yFiles for HTML  KeyLines

- An open-source visualization exploration software without having coding skills
- A tool for data analysts and scientists keen to explore and understand **graphs**
- Functions:
 - Real time visualization
 - Manipulation with Excel structures
 - Appearance properties with metrics
 - Understand patterns in visualization with dynamic filtering and layout
 - Extensible plugins

- Applications of Gephi:
 - Exploratory Data Analysis
 - Link Analysis
 - Social Network Analysis
 - Biological Network Analysis
 - Poster Creation
- Different layouts:
 - CircularLayout
 - GeoLayout
 - Geometric transformation
 - Noverlap
 - OpenOrdLayout

- Prepare:
 - Sample graph: LesMiserables.gexf (download in [here](#) or in class's github)
 - Open Gephi.
 - On the Welcome screen that appears, click on Open Graph File.
 - Open LesMiserables.gexf and click OK



➤ Gephi's interface:

1. Overview: where we can explore the graph visually

2. Data Laboratory: provides an "Excel" table view of the data in network

6. "Appearance", where we can change colors and sizes in interesting ways

3. Preview: where we polish the visualization before exporting it as a picture or pdf

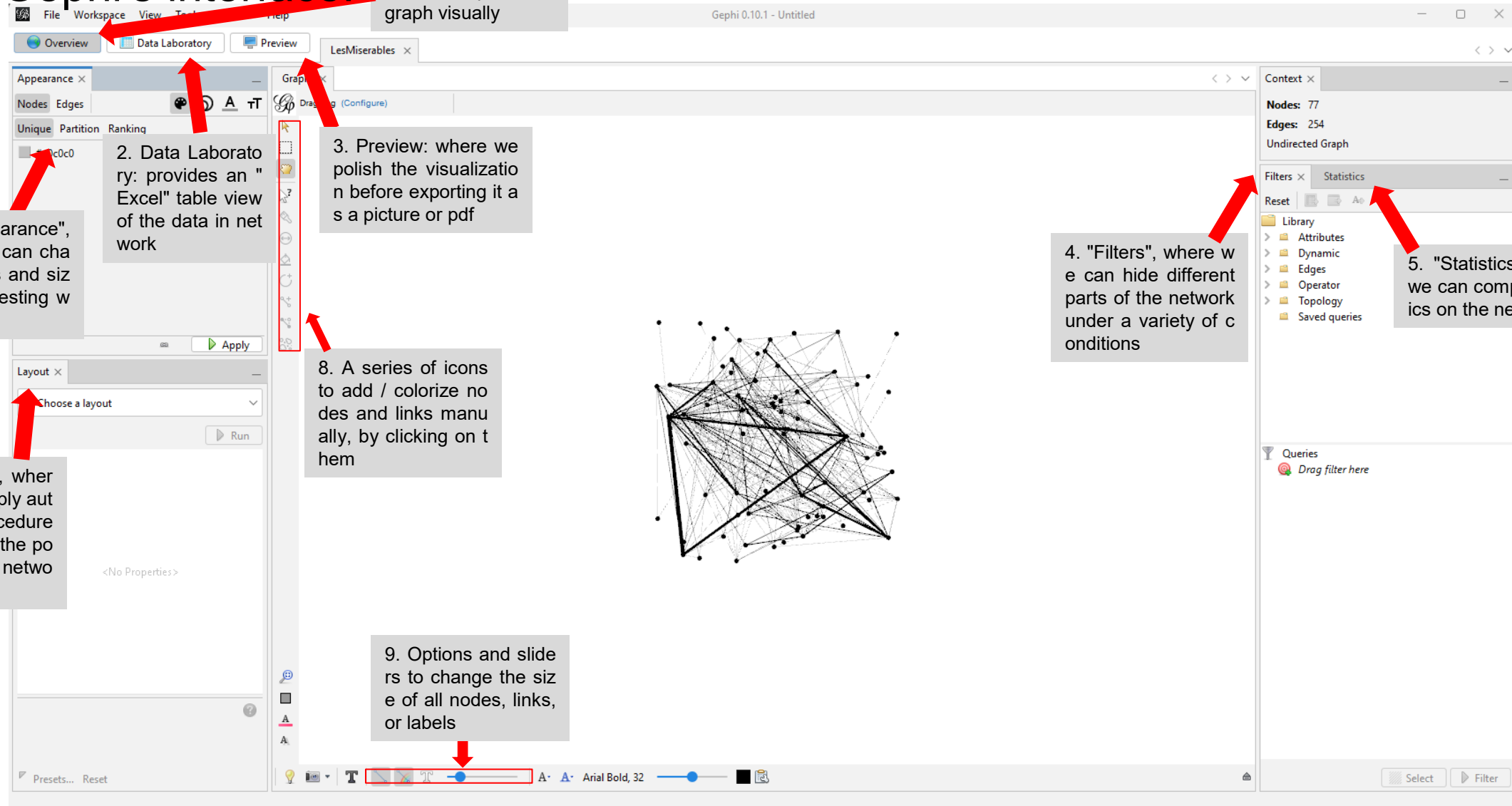
4. "Filters", where we can hide different parts of the network under a variety of conditions

5. "Statistics", where we can compute metrics on the network

8. A series of icons to add / colorize nodes and links manually, by clicking on them

9. Options and sliders to change the size of all nodes, links, or labels

7. "Layouts", where we can apply automated procedures to change the position of the network



➤ Gephi's layout:

Step 1

Step 2: Choose layout

List of layouts

Step 3: Run layout

Step 4: Click on stop to freeze graph

➤ Force atlas 2 layout:

1. Size: change node size.

2. Click Apply for changing the node size

2. Scaling: a parameter to control how "wide" the graph will be.

3. Prevent overlap: a parameter to avoid that nodes are on top of each other. To make it easier to read the network. Check the box.

The screenshot displays the Gephi software interface. On the left, the 'Appearance' panel is open, showing the 'Nodes' tab. The 'Size' parameter is set to 9. Below this, the 'Layout' panel is open, showing the 'ForceAtlas2' layout selected. The 'Performance' section includes 'Tolerance (speed)' at 1.0, 'Approximate Repulsion' (unchecked), and 'Approximation' at 1.2. The 'Tuning' section has 'Scaling' at 10.0, 'Stronger Gravity' (unchecked), and 'Gravity' at 1.0. The 'Behavior Alternatives' section has 'Dissuade Hubs' (unchecked), 'LinLog mode' (unchecked), 'Prevent Overlap' (checked), 'Edge Weight Influence' at 1.0, 'Normalize edge weights' (unchecked), and 'Inverted edge weights' (unchecked). The 'Run' button is visible. On the right, the 'Graph' panel shows a network visualization with nodes and edges. The nodes are labeled with names, and the edges represent connections between them. The layout is a ForceAtlas2 layout, which is a force-directed layout algorithm.

Force Atlas 2 is a layout which:

1. brings together nodes which are connected
2. spreads apart unconnected nodes

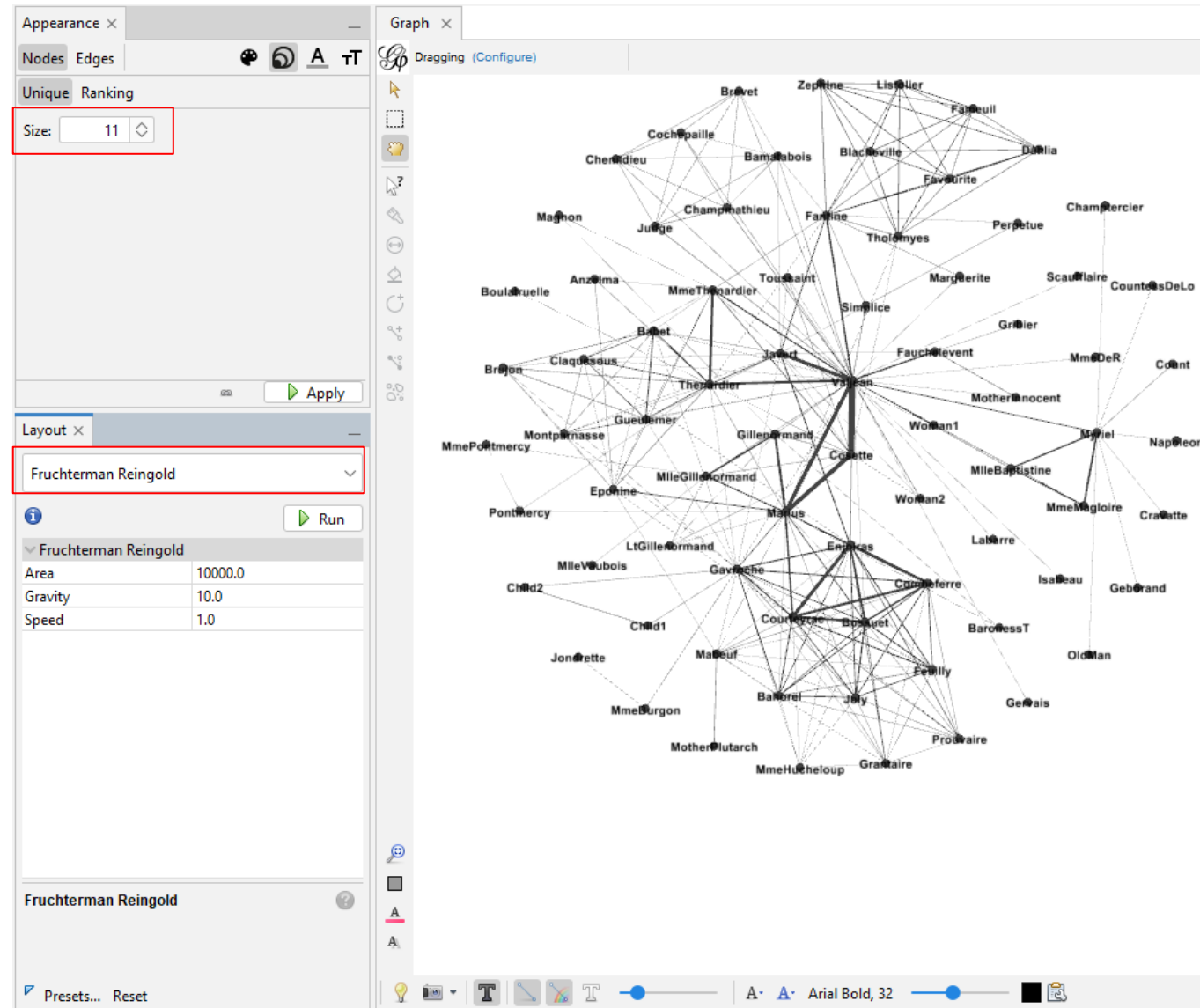
As an effect, we can easily detect communities of nodes

Adjusting the thickness of the links

Change the size of the labels

Show node labels

- Fruchterman and Reingold relies on spring layout



- Circular layout: to use this layout, you need to install new plugin

1. Click on "Tools"

2. Select "Plugins"

3. Search plugin with keyword "circular"

4. Check on the box

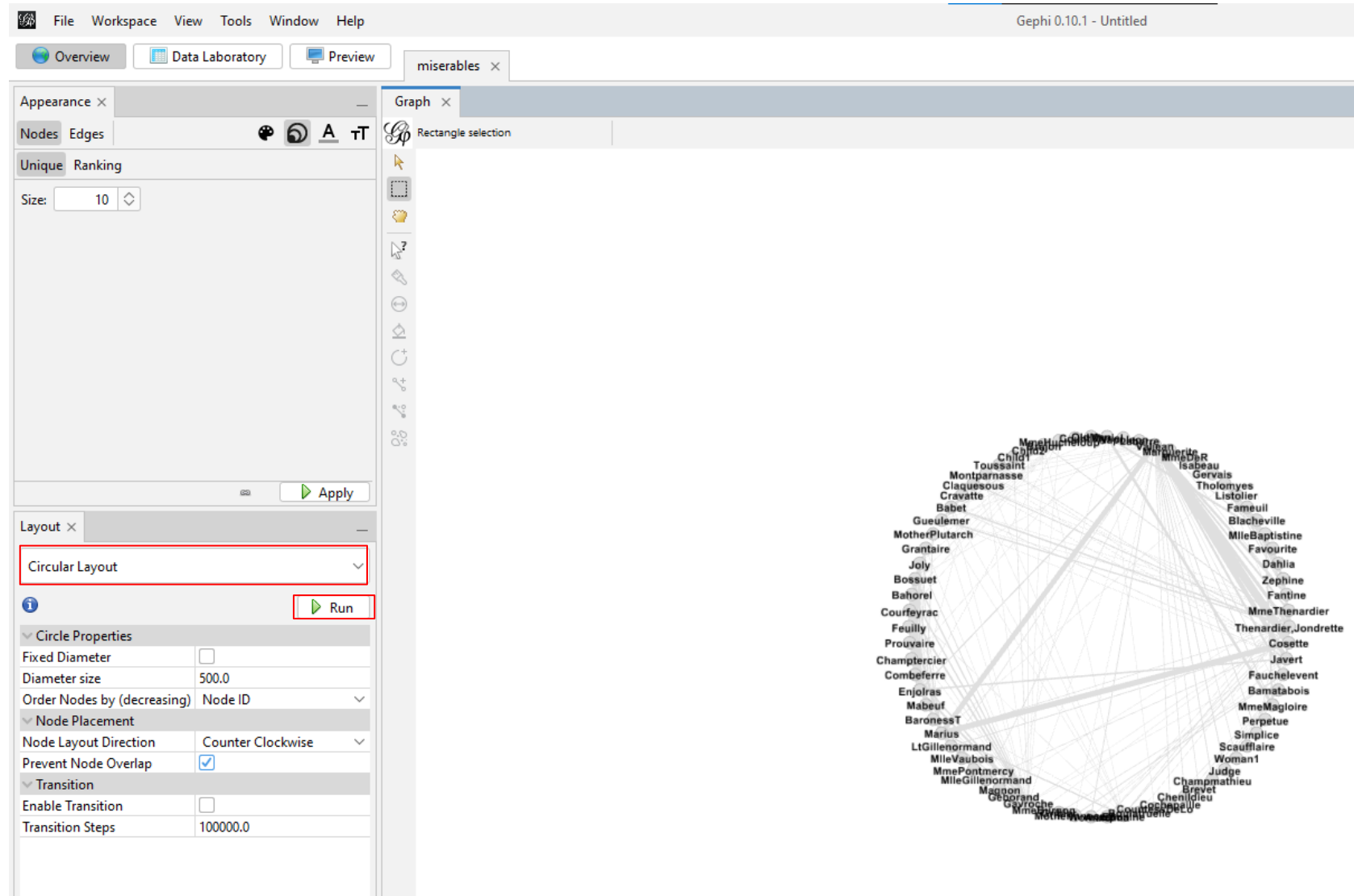
5. Click "Next"

6. Accept the terms of plugin

7. Click "Install"

8. Click "Finish" with restart to apply plugin. Don't forget to save your project before restarting!

➤ Circular layout:



➤ Preview graph:

1. Click "Preview"

2. Select "Presets" to display your graph

3. Custom graph style

4. Click on "Refresh" to apply

Export: SVG/PDF/PNG

Export image

Zoom graph

➤ Switching the view to the data laboratory:

Click on "Data laboratory"

Node labels

Each row contains one node information

	Label	Interval
0.0	Myriel	
1.0	Napoleon	
2.0	MlleBaptistine	
3.0	MmeMagloire	
4.0	CountessDeLo	
5.0	Geborand	
6.0	Champtercier	
7.0	Cravatte	
8.0	Count	
9.0	OldMan	
10.0	Labarre	
11.0	Valjean	
12.0	Marguerite	
13.0	MmeDeR	
14.0	Isabeau	
15.0	Gervais	
16.0	Tholomyes	
17.0	Listolier	
18.0	Fameuil	
19.0	Blacheville	
20.0	Favourite	
21.0	Dahlia	
22.0	Zephine	
23.0	Fantine	
24.0	MmeThenardier	
25.0	Thenardier	
26.0	Cosette	
27.0	Javert	
28.0	Fauchelevant	
29.0	Bamatabois	
30.0	Perpetue	
31.0	Simlice	
32.0	Scaufflaire	
33.0	Woman1	
34.0	Judge	
35.0	Chouanette	

➤ Computing betweenness centrality with Gephi:

1. Click on "Statistics"

2. Run "Network Diameter"

3. Click OK

4. Close

HTML Report Results:
Diameter: 5
Radius: 3
Average Path length: 2.587189929655683

Betweenness Centrality Distribution

Count

Value

Print Copy Save Close

Graph Distance settings

Distance

The average graph-distance between all pairs of nodes. Connected nodes have graph distance 1. The diameter is the longest graph distance between any two nodes in the network. (i.e. How far apart are the two most distant nodes).

☐ Directed ☒ Undirected

☐ Normalize Centralities in [0,1]

Betweenness Centrality: Measures how often a node appears on shortest paths between nodes in the network.

Closeness Centrality: The average distance from a given starting node to all other nodes in the network.

Eccentricity: The distance from a given starting node to the farthest node from it in the network.

OK Cancel

Statistics

Nodes: 74
Edges: 248
Undirected Graph

Filters Statistics

Settings

Network Overview

Avg. Degree 6.703 Run

Weighted Degree Run

Network Diameter Run

Graph Density Run

HITS Run

PageRank Run

Connected Components Run

Community Detection

Modularity Run

Statistical Inference Run

Node Overview

Avg. Clustering Coefficient Run

Eigenvector Centrality Run

Edge Overview

Avg. Path Length Run

Dynamic

Nodes Run

Edges Run

Degree Run

Clustering Coefficient Run

➤ View graph attribute: Betweenness Centrality

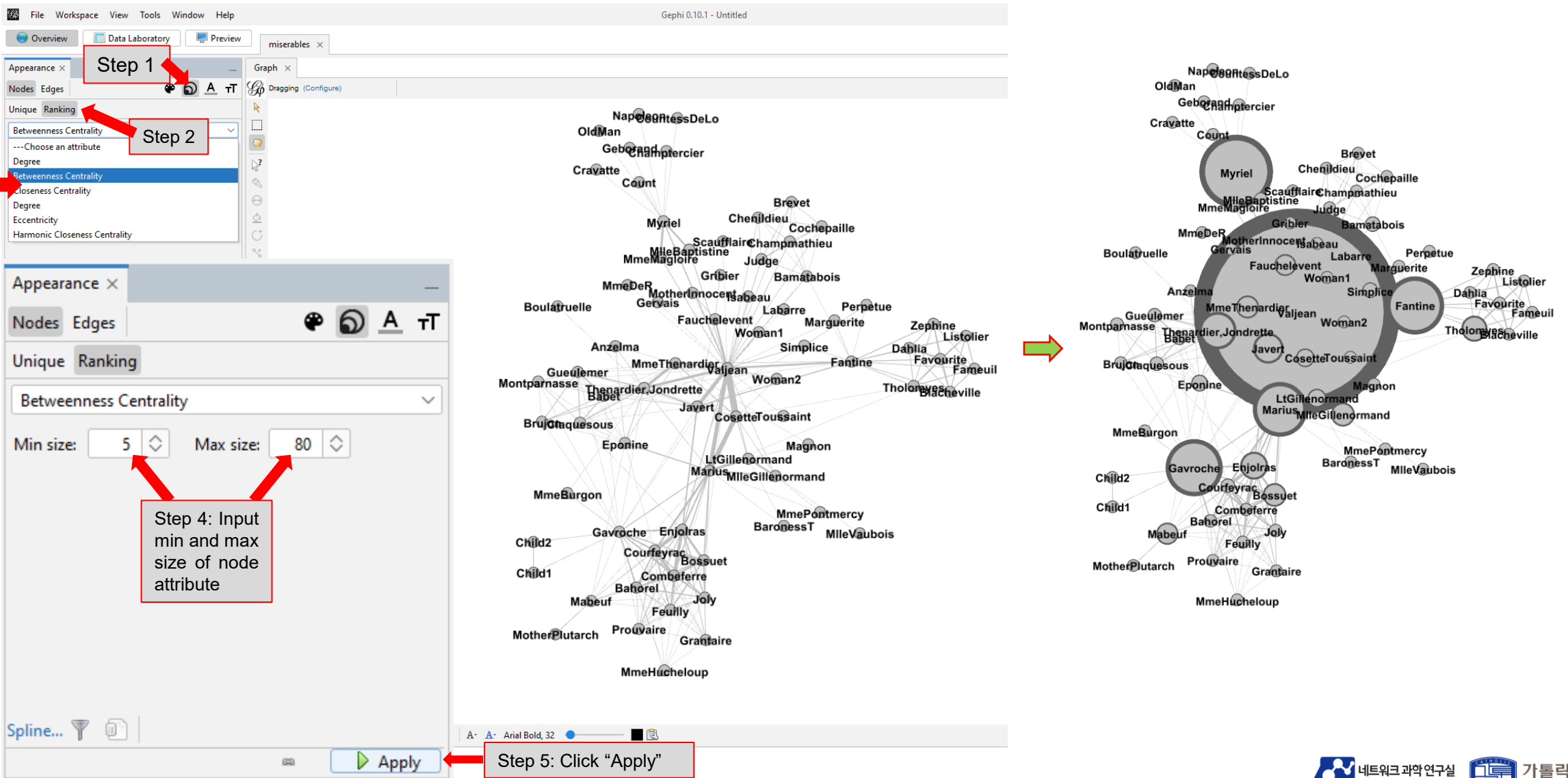
Step 1: Select the 'Data Laboratory' tab and the 'Graph' view.

Step 2: In the 'Ranking' section, select 'Betweenness Centrality' from the dropdown menu.

Step 3: Choose the attribute 'Betweenness Centrality' from the 'Ranking' section.

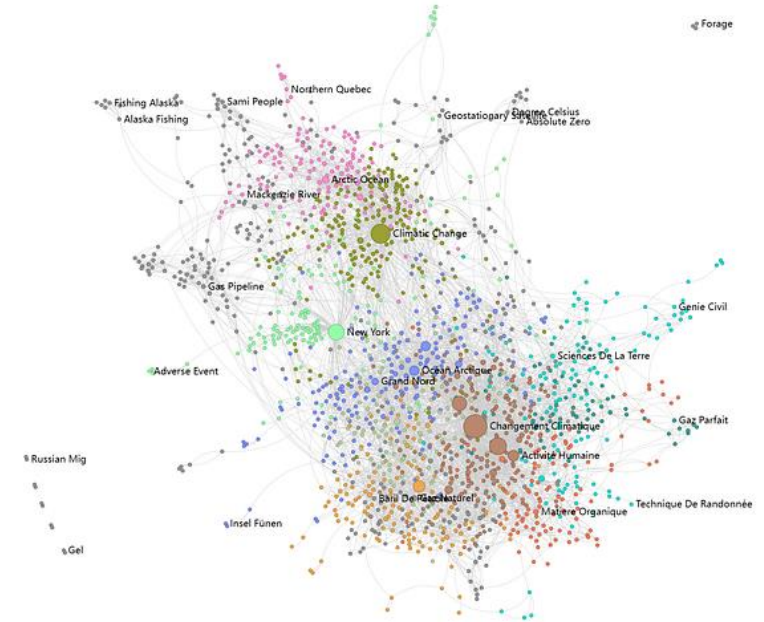
Step 4: Input the minimum size (5) and maximum size (80) for the node attribute.

Step 5: Click the 'Apply' button to visualize the graph with node sizes representing Betweenness Centrality.





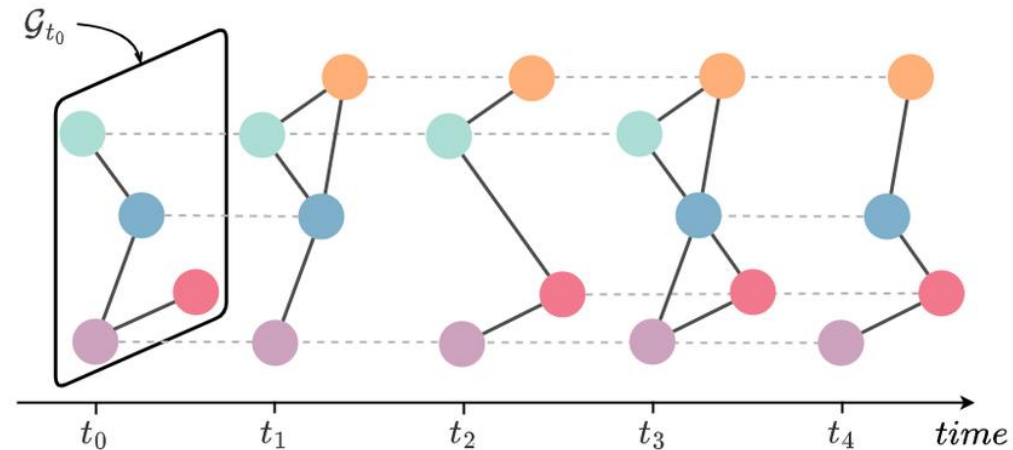
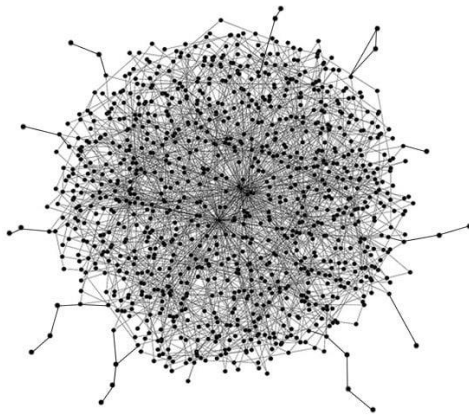
NetworkX visualization



ipysigma visualization

Source: https://github.com/msbased/medium_ipysigma

- **Scalability and “Hairballs”:** Visualizing large graphs (thousands to millions of nodes) often results in cluttered, unreadable layouts known as “hairballs”.
- **Dynamic Data Representation:** Visualizing changing data over time (dynamic graphs) makes it difficult to maintain stability and readability, as nodes move or appear/disappear.
- **Screen Space Limitations:** Even large monitors struggle to display dense graphs effectively, forcing a choice between, or combination of, detailed node-link diagrams and abstract adjacency matrices.
- **Performance Limitations:** Rendering large-scale datasets in real-time requires substantial computational power, pushing the limits of current in-memory technologies.



- **Graph visualization** represents relationships using nodes and edges to reveal patterns and structures
- The **goal** is to create clear, readable layouts by optimizing:
 - Node distance & distribution
 - Edge length & crossings
 - Structural preservation
- **Key layout techniques:**
 - Force-directed (Spring, Fruchterman-Reingold, ForceAtlas2) → reveal clusters & communities
 - Circular layout → highlight symmetry and ordering
 - Tree / hierarchical layouts → represent structured data

- **Tools & ecosystems:**
 - Python: NetworkX, PyVis,...
 - Visualization: Gephi, Cytoscape,...
 - Web & interactive: Plotly, D3.js,...
- **Key challenges:**
 - Large-scale graphs (“**hairball**” problem)
 - Dynamic and temporal data
 - Performance & visualization limits;



네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

